````
# Project4 — RAG Foundation
## Architecture & Developer Documentation

**Version:** 0.1.0 · **Author:** Imran · **Stack:** Python 3.10+, pydantic-settings, LangChain, Ver

---

## 1. Overview

Project4 is the foundation of a Retrieval-Augmented Generation (RAG) application.
It is built with **Clean Architecture (Ports & Adapters / Hexagonal)**:

- **Business logic never touches a vendor SDK.** It depends only on abstract *ports*.
- **Vendor code lives in adapters.** Today: Vercel AI Gateway. Tomorrow: AWS Bedrock, Pinecone.
- Swapping providers is a **one-line `.env` change** — no code changes.

Currently implemented: **LLM chat** and **embeddings**.
Planned next: **Pinecone vector store**, then ingestion & retrieval.

---

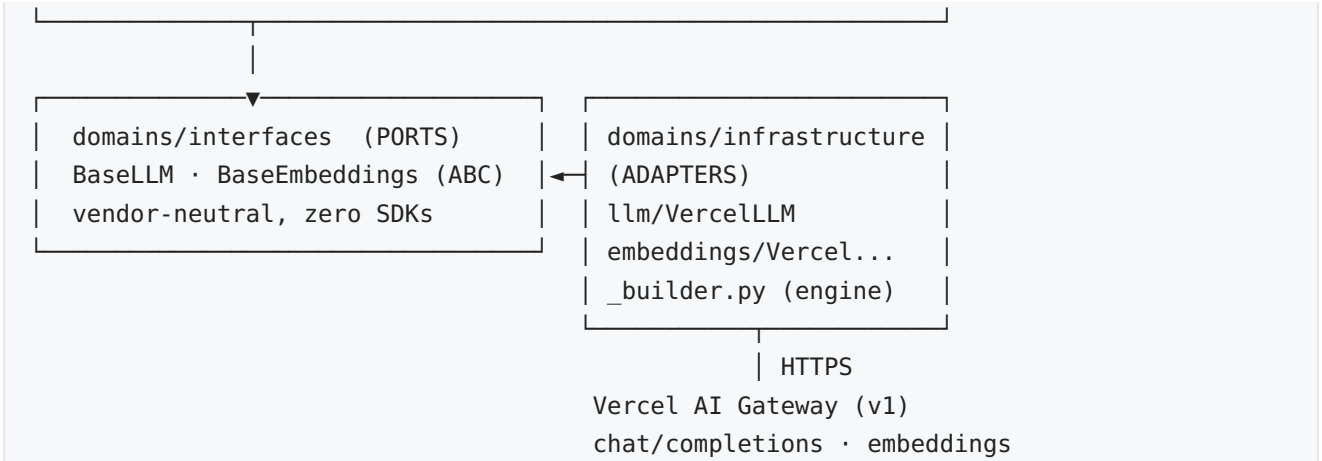
## 2. Tech Stack

| Component | Technology | Role |
|---|---|---|
| Language | Python ≥ 3.10 | Uses `X | None`, `list[...]` generics |
| Settings | pydantic-settings 2.x | Typed `.env` loading, validation, `SecretStr` |
| Models | `pydantic` 2.x | Settings schemas |
| LLM access | langchain-openai (`ChatOpenAI`) | OpenAI-compatible client pointed at the gateway |
| Embeddings | langchain-openai (`OpenAIEmbeddings`) | Same gateway, `/embeddings` endpoint |
| Gateway | Vercel AI Gateway (`https://ai-gateway.vercel.sh/v1`) | One API key → many vendors |
| Vector DB (planned) | Pinecone | Similarity search |
| Packaging | `pip install -e .` via `pyproject.toml` | Makes `app.*` imports work anywhere |

---

## 3. Architecture

```
┌──────────────────────────────────────────────────────────────┐
│                     APPLICATION LAYER                        │
│          app/application/dependencies/providers.py           │
│      (composition root — builds & caches singletons)         │
└──────────────┬───────────────────────────────────────────────┘
               │ calls factories
┌──────────────▼───────────────────────────────────────────────┐
│                     CORE (shared kernel)                     │
│      config.py (typed settings)   exceptions.py (errors)     │
````
````



The dependency rule: `interfaces` depends on nothing → `infrastructure` implements interfaces → `application` wires everything → `core` is used by all, imports nothing above it.

Patterns used

Pattern	Where	Why
Ports & Adapters	`interfaces/` vs `infrastructure/`	Vendor-agnostic business logic
Registry + Factory	`*_factory.py`	Provider selection from `.env`
Singleton	`@lru_cache(maxsize=1)` on providers	One client instance per process
Poor-man's DI	`providers.py`	Any module asks `get_llm()`, never builds its own
Fail-fast config	`get_settings()` + exceptions	Bad config crashes loudly at startup, not mid-

---

4. Directory Structure

```
Project4/
├── src/
│   └── app/
│       ├── __init__.py
│       ├── application/                  # composition root
│       │   └── dependencies/
│       │       └── providers.py          # get_llm / get_embeddings / get_vector_store
│       ├── core/                         # shared kernel
│       │   ├── config.py                 # all settings + get_settings()
│       │   └── exceptions.py             # exception hierarchy
│       └── domains/
│           ├── interfaces/               # PORTS (abstract)
│           │   ├── llm.py
│           │   └── embeddings.py
│           └── infrastructure/           # ADAPTERS (concrete)
│               ├── _builder.py           # shared provider-builder engine
│               ├── llm/
│               │   ├── factory.py
│               │   └── vercel.py
```

```
│               ├── embeddings/
│               │   ├── factory.py
│               │   └── vercel.py
│               └── vector_store/         # empty — Pinecone lands here
├── scripts/
│   └── smoke_test.py                     # end-to-end wiring check
├── docs/
├── tests/
├── pyproject.toml
└── .env                                  # never commit — keep .env in .gitignore
```

---

5. File-by-File Reference

5.1 `app/core/exceptions.py` — the error hierarchy

```python
class AppError(Exception):
    """Base class for all application errors."""

class ConfigError(AppError):
    """Invalid or missing configuration."""

class APIKeyMissingError(ConfigError):
    """A required API key is missing or empty."""

class ProviderError(AppError):
    """A provider could not be resolved or initialised."""

class LLMProviderError(ProviderError): ...
class EmbeddingsProviderError(ProviderError): ...
class VectorStoreProviderError(ProviderError): ...
```

Design decisions
- One base (`AppError`) → the app can catch *anything of its own* with a single `except AppError`.
- `APIKeyMissingError` extends `ConfigError` — a missing key *is* a config problem.
- Separate `LLMProviderError` / `EmbeddingsProviderError` — error messages name the right domain.
- The builder (§5.5) re-raises `AppError` subclasses **unwrapped**, so callers get precise errors,

5.2 `app/core/config.py` — typed settings

```python
from enum import Enum
from functools import lru_cache
from pathlib import Path
from typing import Optional

from pydantic import Field, SecretStr
```

```
from pydantic_settings import BaseSettings, SettingsConfigDict

from app.core.exceptions import ConfigError

BASE_DIR = Path(__file__).resolve().parents[3]   # -> Project4/
ENV_FILE = BASE_DIR / ".env"

class _Base(BaseSettings):
    model_config = SettingsConfigDict(
        env_file=str(ENV_FILE),
        env_file_encoding="utf-8",
        extra="ignore",           # unknown .env vars never crash us
    )

class ProviderEnum(str, Enum):
    VERCEL = "vercel"
    AWS = "aws"
    PINECONE = "pinecone"

class VercelSettings(_Base):
    base_url: str = Field(default="https://ai-gateway.vercel.sh/v1",
                          validation_alias="VERCEL_BASE_URL")
    api_key: Optional[SecretStr] = Field(default=None, validation_alias="VERCEL_API_KEY")

# ... LangChainSettings, PineconeSettings, AWSSettings,
#     ChunkSettings, RetrievalSettings (same pattern)

class AppSettings(_Base):
    embedding_provider: ProviderEnum = Field(default=ProviderEnum.VERCEL, validation_alias="EMBEDDI
    embedding_model: str = Field(default="cohere/embed-v4.0", validation_alias="EMBEDDING_MODEL")
    embedding_dimensions: int = Field(default=1024, validation_alias="EMBEDDING_DIMENSIONS")
    llm_provider: ProviderEnum = Field(default=ProviderEnum.VERCEL, validation_alias="LLM_PROVIDER"
    llm_model: str = Field(default="openai/gpt-4o-mini", validation_alias="LLM_MODEL")
    vercel: VercelSettings = Field(default_factory=VercelSettings)
    langchain: LangChainSettings = Field(default_factory=LangChainSettings)
    database_url: Optional[SecretStr] = Field(default=None, validation_alias="DATABASE_URL")
    pinecone: PineconeSettings = Field(default_factory=PineconeSettings)
    aws: AWSSettings = Field(default_factory=AWSSettings)
    chunk: ChunkSettings = Field(default_factory=ChunkSettings)
    retrieval: RetrievalSettings = Field(default_factory=RetrievalSettings)

@lru_cache(maxsize=1)
def get_settings() -> AppSettings:
    try:
        return AppSettings()
    except Exception as e:
        raise ConfigError(f"Invalid configuration: {e}") from e
```

Key points
- `parents[3]`: the file sits at `src/app/core/config.py` → `parents[0]`=core, `[1]`=app, `[2]`=src

- `validation_alias` maps uppercase env names to lowercase Python attributes.
- `SecretStr` hides keys in logs/repr; read via `.get_secret_value()`.
- **Nested settings** (`default_factory=VercelSettings`): each child reads the same `.env` independ
- Integrations not wired yet (AWS/Pinecone/DB) are `Optional` → the app **boots today**; their adap
- `get_settings()` is **lazy and cached**: no import-time crashes, one parse per process, invalid c

5.3 `app/domains/interfaces/llm.py` — the LLM port

```python
from abc import ABC, abstractmethod
from typing import TypedDict

class Message(TypedDict):
    role: str        # "system" | "user" | "assistant"
    content: str

class BaseLLM(ABC):
    @abstractmethod
    def generate(self, messages: list[Message]) -> str: ...
```

The **only** contract business logic sees. Plain dicts, no LangChain types, no SDK types — a port m

5.4 `app/domains/interfaces/embeddings.py` — the embeddings port

```python
from abc import ABC, abstractmethod

class BaseEmbeddings(ABC):
    @abstractmethod
    def embed_documents(self, texts: list[str]) -> list[list[float]]: ...
    @abstractmethod
    def embed_query(self, text: str) -> list[float]: ...
```

Two methods, matching the industry convention: `embed_documents` for corpus ingestion (batch), `emb

5.5 `app/domains/infrastructure/_builder.py` — the shared factory engine

```python
from functools import reduce
from typing import Optional, Type, TypeVar

from app.core.config import AppSettings, get_settings
from app.core.exceptions import APIKeyMissingError, AppError, ProviderError

T = TypeVar("T")

def resolve_dotted(obj: object, dotted: str):
    """'vercel.api_key' -> obj.vercel.api_key (works with nested settings)."""
    return reduce(getattr, dotted.split("."), obj)
```

```
def build_provider(
    *,
    registry: dict[str, Type[T]],
    provider: str,
    kind: str,
    api_key_path: Optional[str],
    error: Type[ProviderError],
    settings: Optional[AppSettings] = None,
) -> T:
    settings = settings or get_settings()

    adapter_cls = registry.get(provider)
    if adapter_cls is None:
        raise error(
            f"Unknown {kind} provider '{provider}'. "
            f"Available: {', '.join(registry)}. Check your .env."
        )

    if api_key_path:
        key = resolve_dotted(settings, api_key_path)
        get_secret = getattr(key, "get_secret_value", None)
        if key is None or (get_secret is not None and not get_secret()):
            raise APIKeyMissingError(
                f"'{api_key_path}' is missing or empty for {kind} provider '{provider}'."
            )

    try:
        return adapter_cls(settings)
    except AppError:
        raise                       # already meaningful — don't wrap
    except Exception as e:
        raise error(f"Failed to initialise {kind} provider '{provider}': {e}") from e
```

What it does, in order: (1) registry lookup → `ProviderError` listing valid options; (2) API-ke

5.6 `app/domains/infrastructure/llm/vercel.py` — LLM adapter

```python
from langchain_core.messages import convert_to_messages
from langchain_openai import ChatOpenAI

class VercelLLM(BaseLLM):
    def __init__(self, settings: AppSettings) -> None:
        api_key = settings.vercel.api_key
        if not api_key or not api_key.get_secret_value():
            raise APIKeyMissingError("VERCEL_API_KEY is empty. Add it to your .env.")
        self._llm = ChatOpenAI(
            model=settings.llm_model,          # slug decides the vendor
            api_key=api_key,
```

```
            base_url=settings.vercel.base_url, # https://ai-gateway.vercel.sh/v1
            temperature=0.1,
        )

    def generate(self, messages: list[Message]) -> str:
        response = self._llm.invoke(convert_to_messages(messages))
        return str(response.content)
```

- **One adapter serves every vendor** — `'openai/gpt-4o-mini'`, `'anthropic/claude-...'`, `'cohere/
- Key is checked again at the adapter (defense in depth).
- `convert_to_messages()` maps our plain-dict `Message` → LangChain message objects.
- `temperature=0.1` → mostly deterministic output for a retrieval product.

5.7 `app/domains/infrastructure/llm/factory.py` — LLM registry

```python
LLM_REGISTRY: dict[str, type[BaseLLM]] = {
    "vercel": VercelLLM,
    # "aws_bedrock": BedrockLLM,      # <- future adapter goes here
}
API_KEY_REQUIREMENTS: dict[str, str] = {"vercel": "vercel.api_key"}

def build_llm(settings: Optional[AppSettings] = None) -> BaseLLM:
    s = settings or get_settings()
    provider = s.llm_provider.value.lower()
    return build_provider(
        registry=LLM_REGISTRY, provider=provider, kind="LLM",
        api_key_path=API_KEY_REQUIREMENTS.get(provider),
        error=LLMProviderError, settings=s,
    )
```

5.8 `app/domains/infrastructure/embeddings/vercel.py` — embeddings adapter

```python
from langchain_openai import OpenAIEmbeddings

class VercelEmbeddings(BaseEmbeddings):
    def __init__(self, settings: AppSettings) -> None:
        api_key = settings.vercel.api_key
        if not api_key or not api_key.get_secret_value():
            raise APIKeyMissingError("VERCEL_API_KEY is empty. Add it to your .env.")
        self._embeddings = OpenAIEmbeddings(
            model=settings.embedding_model,        # cohere/embed-v4.0
            api_key=api_key,
            base_url=settings.vercel.base_url,     # .../v1 -> /embeddings
            dimensions=settings.embedding_dimensions,
            check_embedding_ctx_length=False,
        )
```

```
    def embed_documents(self, texts): return self._embeddings.embed_documents(texts)
    def embed_query(self, text):      return self._embeddings.embed_query(text)
```

- `dimensions=1024` — Cohere embed-v4 supports variable output dims. **This number MUST equal the P
- `check_embedding_ctx_length=False` — stops LangChain from tokenising non-OpenAI models with tikto

5.9 `app/domains/infrastructure/embeddings/factory.py`

Identical pattern to §5.7 with `EMBEDDINGS_REGISTRY`, `EMBEDDING_PROVIDER`, and `EmbeddingsProvider

5.10 `app/application/dependencies/providers.py` — composition root

```python
from functools import lru_cache

@lru_cache(maxsize=1)
def get_llm() -> BaseLLM:            return build_llm()

@lru_cache(maxsize=1)
def get_embeddings() -> BaseEmbeddings: return build_embeddings()

@lru_cache(maxsize=1)
def get_vector_store():
    raise NotImplementedError("Pinecone adapter comes next")

def reset_all_caches() -> None:
    for fn in (get_llm, get_embeddings, get_vector_store):
        fn.cache_clear()
```

- `@lru_cache(maxsize=1)` → **singleton**: the first caller builds the client, everyone after reuse
- `reset_all_caches()` → used in tests when env vars are monkeypatched.
- The `__main__` smoke test moved to `scripts/smoke_test.py` — the composition root stays wiring-or

---

6. Request Flow (what happens on `python -m scripts.smoke_test`)

1. `get_llm()` → `lru_cache` miss → `build_llm()`
2. `get_settings()` → cached `AppSettings` parses `.env` (or raises `ConfigError`)
3. `LLM_PROVIDER=vercel` → `LLM_REGISTRY["vercel"]` → `VercelLLM`
4. `build_provider` checks `vercel.api_key` → empty? → `APIKeyMissingError`
5. `VercelLLM.__init__` → `ChatOpenAI(model=..., base_url=gateway)`
6. `generate()` → `convert_to_messages` → `POST /v1/chat/completions`
7. Response content returned as `str`; **instance cached** — later calls skip 2–5
8. Same chain for embeddings via `POST /v1/embeddings`

---

7. Environment Variables Reference

````
| Variable | Default | Required today | Purpose |
|---|---|---|---|
| `VERCEL_API_KEY` | — | ✅ | Auth for AI Gateway (chat + embeddings) |
| `VERCEL_BASE_URL` | `https://ai-gateway.vercel.sh/v1` | — | Gateway endpoint |
| `LLM_PROVIDER` | `vercel` | — | Key into `LLM_REGISTRY` |
| `LLM_MODEL` | `openai/gpt-4o-mini` | — | Vendor/model slug |
| `EMBEDDING_PROVIDER` | `vercel` | — | Key into `EMBEDDINGS_REGISTRY` |
| `EMBEDDING_MODEL` | `cohere/embed-v4.0` | — | Embedding model slug |
| `EMBEDDING_DIMENSIONS` | `1024` | — | Must match Pinecone index dimension |
| `LANGCHAIN_TRACING_V2` | `false` | — | LangSmith tracing toggle |
| `LANGCHAIN_API_KEY` | — | — | LangSmith key |
| `DATABASE_URL` | — | — | Future use |
| `PINECONE_API_KEY` / `_INDEX_HOST` / `_DIMENSION` / `_CLOUD` / `_REGION` / `_ENVIRONMENT` / `_REA
| `AWS_ACCESS_KEY_ID` / `AWS_SECRET_ACCESS_KEY` / `AWS_REGION` / `AWS_S3_BUCKET_NAME` | — | — | Fut
| `CHUNK_SIZE` / `CHUNK_OVERLAP` | `500` / `50` | — | Future ingestion |
| `DEFAULT_RETRIEVAL_K` / `HYBRID_FETCH_K` | `4` / `20` | — | Future retrieval |
| `USE_RERANKER` / `RERANKER_MODEL` / `SIMILARITY_THRESHOLD` | `true` / `""` / `0.70` | — | Future

---

## 8. Usage Examples

```python
from app.application.dependencies.providers import get_llm, get_embeddings

# Chat
llm = get_llm()
print(llm.generate([
    {"role": "system", "content": "You are terse."},
    {"role": "user", "content": "Explain RAG in one sentence."},
]))

# Embeddings
embs = get_embeddings()
vectors = embs.embed_documents(["doc one", "doc two"])   # batch for ingestion
query_vec = embs.embed_query("what is rag?")             # single search vector
print(len(query_vec))                                    # 1024
```

---

## 9. How to Add a New Provider (e.g., OpenAI direct)

1. **Adapter** — implement the port, nothing else:

   ```python
   # app/domains/infrastructure/llm/openai_direct.py
   from langchain_openai import ChatOpenAI
   from app.core.config import AppSettings
   from app.core.exceptions import APIKeyMissingError
````
````

```
   from app.domains.interfaces.llm import BaseLLM, Message

   class OpenAIDirectLLM(BaseLLM):
       def __init__(self, settings: AppSettings) -> None:
           if not settings.openai.api_key:
               raise APIKeyMissingError("OPENAI_API_KEY is empty.")
           self._llm = ChatOpenAI(model=settings.llm_model,
                                  api_key=settings.openai.api_key,
                                  temperature=0.1)
       def generate(self, messages: list[Message]) -> str:
           return str(self._llm.invoke(convert_to_messages(messages)).content)
   ```

2. **Settings** — add an `OpenAISettings(_Base)` section + field on `AppSettings`.
3. **Register** — `LLM_REGISTRY["openai"] = OpenAIDirectLLM`
4. **Key path** — `API_KEY_REQUIREMENTS["openai"] = "openai.api_key"`
5. **Switch** — set `LLM_PROVIDER=openai` in `.env`. Done — zero changes anywhere else.

The same 5 steps apply for embeddings and (later) vector stores.

---

10. Error Reference

Exception	Raised when	Fix
`ConfigError`	`.env` missing/invalid values	Check variable names & values
`APIKeyMissingError`	Chosen provider's key is empty	Add e.g. `VERCEL_API_KEY`
`LLMProviderError`	`LLM_PROVIDER` not in registry / init failed	Use a listed provider; see ch
`EmbeddingsProviderError`	Same, for embeddings	Same
`NotImplementedError`	`get_vector_store()` called	Expected — Pinecone adapter is next
`ModuleNotFoundError: app.*`	Ran from wrong dir / not installed	`pip install -e .` or run fro
`ImportError ... (unknown location)`	Stale `__pycache__`/shadow dir	`find src -type d -name _

---

11. Running & Testing

```bash
pip install -e .                              # one-time, from Project4 root
python -m scripts.smoke_test                  # LLM + embeddings end-to-end
pytest                                        # self-skips without VERCEL_API_KEY
```

Smoke test (`scripts/smoke_test.py`) verifies both adapters:

```python
from app.application.dependencies.providers import get_embeddings, get_llm

def check_llm():
    reply = get_llm().generate([
```

```
        {"role": "system", "content": "You are terse."},
        {"role": "user", "content": "Say hello in exactly 3 words."},
    ])
    print(f"[llm]        OK -> {reply!r}")

def check_embeddings():
    vec = get_embeddings().embed_query("hello world")
    print(f"[embeddings] OK -> dims={len(vec)}, first3={vec[:3]}")

if __name__ == "__main__":
    check_llm(); check_embeddings()
```

---

12. Roadmap

Phase	Deliverable
✅ Done	Ports, adapters, factories, settings, providers, smoke test
➡️ Next	`interfaces/vector_store.py` port + `infrastructure/vector_store/pinecone.py` + factory
Then	Ingestion pipeline: load → clean → chunk (`CHUNK_*`) → embed → upsert
Then	Retrieval: query embed → Pinecone query (`DEFAULT_RETRIEVAL_K`, `SIMILARITY_THRESHOLD`) →
Later	AWS S3 documents, Bedrock LLM, hybrid search (`HYBRID_FETCH_K`)

Port to implement next:

```python
# app/domains/interfaces/vector_store.py
from abc import ABC, abstractmethod
from typing import Any

class BaseVectorStore(ABC):
    @abstractmethod
    def upsert(self, ids: list[str], vectors: list[list[float]],
               metadatas: list[dict[str, Any]]) -> None: ...
    @abstractmethod
    def query(self, vector: list[float], k: int = 4) -> list[dict[str, Any]]: ...
```

---

13. Lessons Learned (keep these!)

1. **Absolute imports only** — every import starts with `app.`, never `from .app...` (a stray dot t
2. **Never run files directly** (`python src/app/...py`) — always `python -m app...` or rely on the
3. **After refactors, nuke caches** — leftover `__pycache__` makes `rmdir` fail silently and dead c
4. **`EMBEDDING_DIMENSIONS` must equal the Pinecone index dimension** — a mismatch fails at upsert
5. **`check_embedding_ctx_length=False`** for non-OpenAI embedding models behind an OpenAI-compatib
6. **Config is lazy** (`get_settings()`) — never instantiate settings at import time; import-time c

````
---

## Appendix A — `.env` template

```env
# ---- required today ----
VERCEL_API_KEY=your_key_here

# ---- optional (defaults shown) ----
LLM_PROVIDER=vercel
LLM_MODEL=openai/gpt-4o-mini
EMBEDDING_PROVIDER=vercel
EMBEDDING_MODEL=cohere/embed-v4.0
EMBEDDING_DIMENSIONS=1024
VERCEL_BASE_URL=https://ai-gateway.vercel.sh/v1

# LANGCHAIN_TRACING_V2=true
# LANGCHAIN_API_KEY=

# ---- future: pinecone ----
# PINECONE_API_KEY=
# PINECONE_INDEX_HOST=
# PINECONE_DIMENSION=1024

# ---- future: aws ----
# AWS_ACCESS_KEY_ID=
# AWS_SECRET_ACCESS_KEY=
# AWS_REGION=us-east-1
# AWS_S3_BUCKET_NAME=

# ---- future: app ----
# DATABASE_URL=
```

## Appendix B — `pyproject.toml`

```toml
[project]
name = "project4"
version = "0.1.0"
requires-python = ">=3.10"
dependencies = [
    "pydantic>=2.7",
    "pydantic-settings>=2.3",
    "langchain-core>=0.3",
    "langchain-openai>=0.2",
]

[build-system]
requires = ["setuptools>=68"]
build-backend = "setuptools.build_meta"
````
````

```
[tool.setuptools.packages.find]
where = ["src"]
```

Appendix C — Command Cheat Sheet

Task	Command (from `Project4/`)
Install	`pip install -e .`
Smoke test	`python -m scripts.smoke_test`
Run providers directly	`python -m app.application.dependencies.providers`
Run pytest	`pytest`
Reset caches after refactor	`find src -type d -name __pycache__ -exec rm -rf {} +`
Inspect settings	`python -c "from app.core.config import get_settings; s=get_settings(); print(